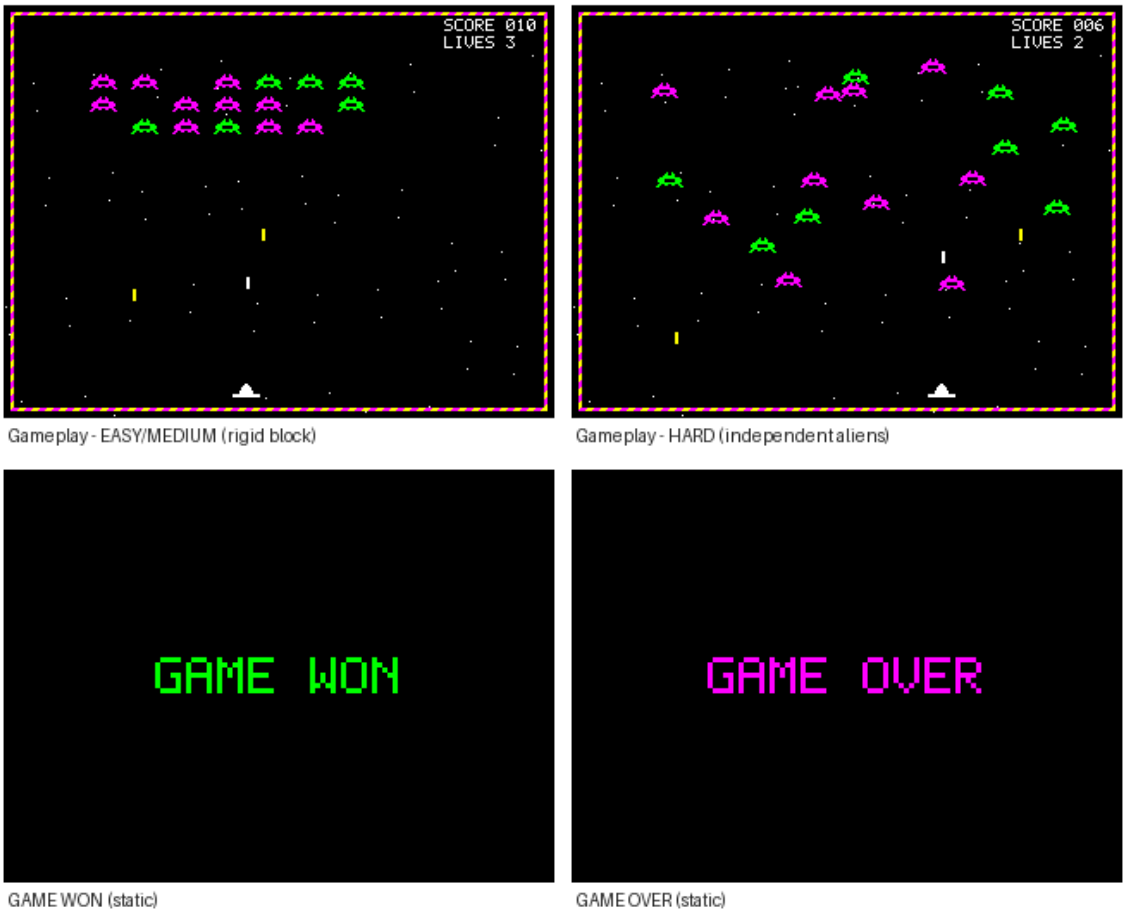
Space Invaders on the DE2-115

A complete walk-through of the Verilog design

Verilog reproduction and extension of “Design and Implementation of a FPGA based Game ‘Space Invaders’” (Parihar, Kumar, Mishra, IJCTA Vol. 9 No. 44, 2016)
Target: Terasic DE2-115, Cyclone IV E EP4CE115F29C7, VGA 640x480 @ ~60 Hz
Course project: Digital Design with FPGA



Gameplay - EASY/MEDIUM (rigid block) | Gameplay - HARD (independent aliens)

GAME WON (static) | GAME OVER (static)

Figure 0. The four kinds of screen the design produces (images generated from the design's own bitmaps and colour rules).

How to read this report. Every section explains one block of the design, shows a diagram, and then quotes the *actual* Verilog being explained (file name and line numbers are given under each listing, so you can open the file and follow along). Section 18 is a glossary that defines every signal, parameter and abbreviation used. All code excerpts are taken directly from the project source; nothing is retyped.

Status note. The design has been checked with a lint script and a cycle-level Python model of the HARD-mode motion, but it has not been compiled in Quartus or simulated in ModelSim from this environment. The first compile on the board is the real test.

1. What the project is

The paper describes a hardware-only Space Invaders game: there is no processor and no software. The whole game is a set of flip-flops and gates that (a) generates the VGA video signal, (b) keeps the positions of the ship, the aliens and the bullets in registers, and (c) for every pixel of every frame decides which colour that pixel must be. This project rebuilds that idea in Verilog for the DE2-115 board and extends it well beyond the paper.

Feature	Paper	This project
Video	VGA, one alien row, 3 colours	VGA 640x480, 3 rows x 7 = 21 aliens, 8-colour palette, starfield and striped border
Game flow	Plays immediately	Title screen -> play -> GAME OVER or GAME WON (4-state FSM)
Difficulty	One	EASY, MEDIUM (rigid random block), HARD (21 independent aliens), chosen by SW[1:0]
Alien fire	None in the paper's code	Two independent random shooters (A and B)
Lives	None	3 lives (3-2-1-0); an alien reaching the ship's row ends the game at once
Score	Debug signal	On-screen SCORE/LIVES text plus 7-segment display (score in HEX0/HEX1, level in HEX2)
Randomness	None	Two free-running 32-bit LFSRs and a 16-bit LFSR inside every HARD alien

Controls on the board

Board item	Function
KEY0	Reset: return to the title screen from any state (clears score, lives, aliens, bullets)
KEY1 / KEY2	Move ship right / left (also start the game from the title screen)
KEY3	Shoot (also starts the game from the title screen)
SW[1:0]	Mode, read on the title screen: both down = EASY, SW0 up = MEDIUM, SW1 up = HARD (SW1 wins)
HEX0, HEX1, HEX2	Score (two hex digits) and level

2. Architecture and files

The design is a small hierarchy. The board-level wrapper **de2115_top** conditions the inputs (debouncing, synchronising) and maps the 3-bit colour to the VGA pins. The core module **main** keeps the paper's original eight-port interface and instantiates three blocks: a VGA timing generator, a slow-tick divider and the big **graphics** module that holds the whole game. A single output register (“fd_1” in the paper's Figure 4) sits between the colour mux and the pins.

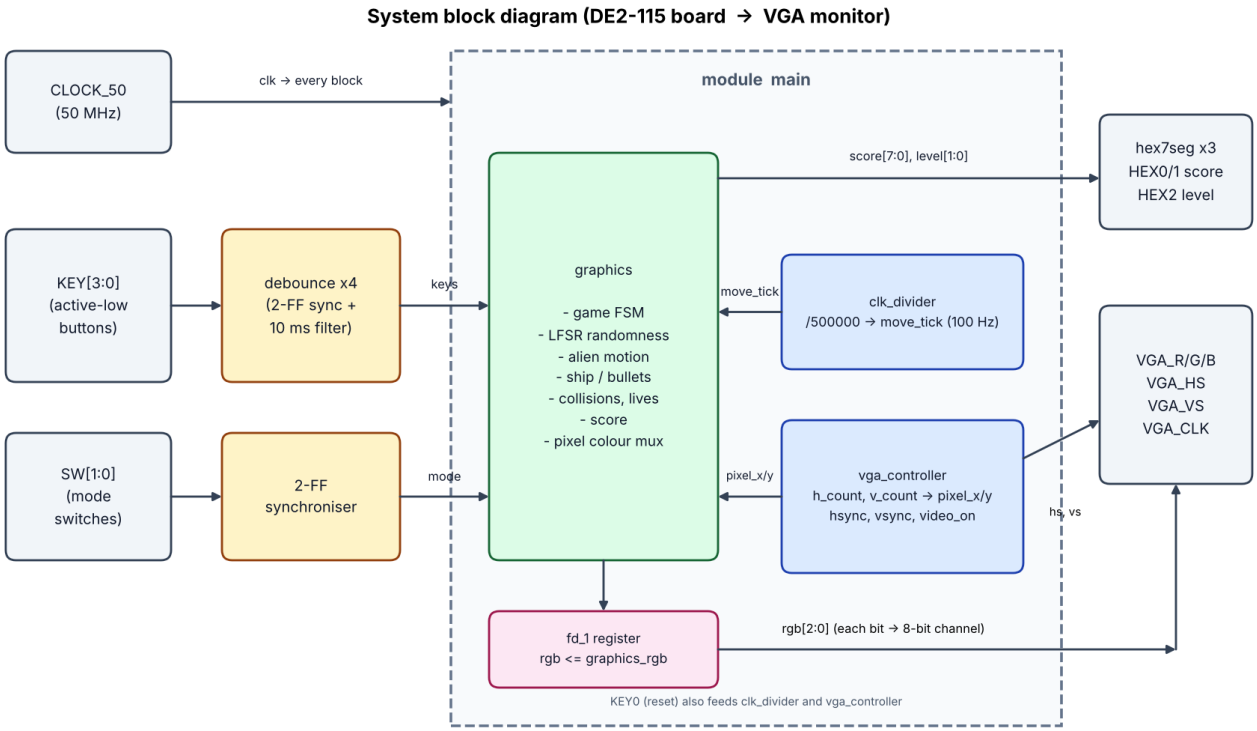


Figure 1. Block diagram. Every flip-flop in the design is clocked by the single 50 MHz board clock.

File	Role	Lines
board/de2115/de2115_top.v	Top level: debounce keys, synchronise switches, colour -> VGA pins, 7-segment drivers	139
board/de2115/de2115_pins.qsf	Quartus pin assignments and device (EP4CE115F29C7)	152
src/main.v	Paper-faithful core: wires vga_controller, clk_divider, graphics and the output register	109
src/vga_controller.v	640x480@60 Hz raster counters, sync pulses, video_on	89
src/clk_divider.v	Generates the ~100 Hz move_tick enable pulse	48
src/graphics.v	The game: FSM, motion, collisions, lives, score, text, sprites, pixel colour mux	1480
src/debounce.v	2-flop synchroniser + 10 ms filter for the push buttons	46
src/hex7seg.v	Hex digit -> 7-segment pattern (active low)	45
sim/tb_main.v	Five testbenches (basic, touch, lives, modes, HARD)	774

3. Input conditioning

3.1 Push buttons: debounce.v

A mechanical button does not make a clean 0-to-1 step: the contact chatters for a few milliseconds. Besides, the button is *asynchronous* to the 50 MHz clock, so sampling it directly could put a flip-flop into a metastable state. The **debounce** module first passes the raw pin through two flip-flops (**in_sync0**, **in_sync1**) to synchronise it, and then only accepts a new level once it has stayed different from the current output for DEBOUNCE_LIMIT = 500,000 clocks (10 ms at 50 MHz).

src/debounce.v (src/debounce.v, lines 1-46, comment-only lines removed)

```
`timescale 1ns / 1ps
module debounce #(
    parameter integer DEBOUNCE_LIMIT = 500000   // ~10 ms at 50 MHz
) (
    input  wire clk,
    input  wire in,
    output reg  out = 1'b1   // idle/un-pressed level for DE2-115's active-low KEYs
);
    reg in_sync0 = 1'b1;
    reg in_sync1 = 1'b1;
    reg [19:0] counter = 20'd0;
    always @(posedge clk) begin
        in_sync0 <= in;
        in_sync1 <= in_sync0;
    end
    always @(posedge clk) begin
```

```
        if (in_sync1 == out) begin
            counter <= 20'd0;
        end else begin
            counter <= counter + 20'd1;
            if (counter >= DEBOUNCE_LIMIT) begin
                out     <= in_sync1;
                counter <= 20'd0;
            end
        end
    end
endmodule
```

The DE2-115 keys are **active low** (0 when pressed), so the output idles at 1. de2115_top inverts the three game keys so that inside the game “1” means “pressed”, while KEY0 is used as it is: the design's reset input is called **not_reset** (reverse-logic reset, as in the paper): 0 = reset.

board/de2115/de2115_top.v (board/de2115/de2115_top.v, lines 54-59, 64-66, comment-only lines removed)

```
wire [3:0] key_d;
debounce U_DEB0 (.clk(CLOCK_50), .in(KEY[0]), .out(key_d[0]));
debounce U_DEB1 (.clk(CLOCK_50), .in(KEY[1]), .out(key_d[1]));
debounce U_DEB2 (.clk(CLOCK_50), .in(KEY[2]), .out(key_d[2]));
debounce U_DEB3 (.clk(CLOCK_50), .in(KEY[3]), .out(key_d[3]));
...
wire right = ~key_d[1];
wire left  = ~key_d[2];
wire shoot = ~key_d[3];
```

3.2 Mode switches: a synchroniser only

A slide switch does not bounce, and the mode is only read at the instant a game key is pressed, so no 10 ms filter is needed. The switches are asynchronous, though, so each goes through a 2-flop synchroniser. (An earlier version reused the debounce module for them, which wasted about 40 flip-flops; it was replaced by this.) SW[1] has priority: if both switches are up the mode is HARD.

board/de2115/de2115_top.v (board/de2115/de2115_top.v, lines 68-77)

```
// SW[1:0] pick the mode: SW[1] up = HARD (it wins over SW[0]), else
// SW[0] up = MEDIUM, both down = EASY. A slide switch doesn't bounce, so
// no debounce counter is needed -- but it is asynchronous to CLOCK_50, so
// it still goes through a 2-flop synchroniser (avoids metastability).
reg [1:0] sw_s0 = 2'b00, sw_d = 2'b00;
always @(posedge CLOCK_50) begin
    sw_s0 <= SW;
    sw_d  <= sw_s0;
end
wire [1:0] mode = {sw_d[1], sw_d[0] & ~sw_d[1]};   // 00 EASY, 01 MEDIUM, 10 HARD
```

4. Clocks and timing

The design never creates a second clock. Everything runs from **CLOCK_50**. Slower rates are produced as *one-clock-wide enable pulses* that are checked with an **if** inside the normal clocked blocks. This is the recommended FPGA style: one clock domain, no clock skew problems, and the timing tool analyses one clock only.

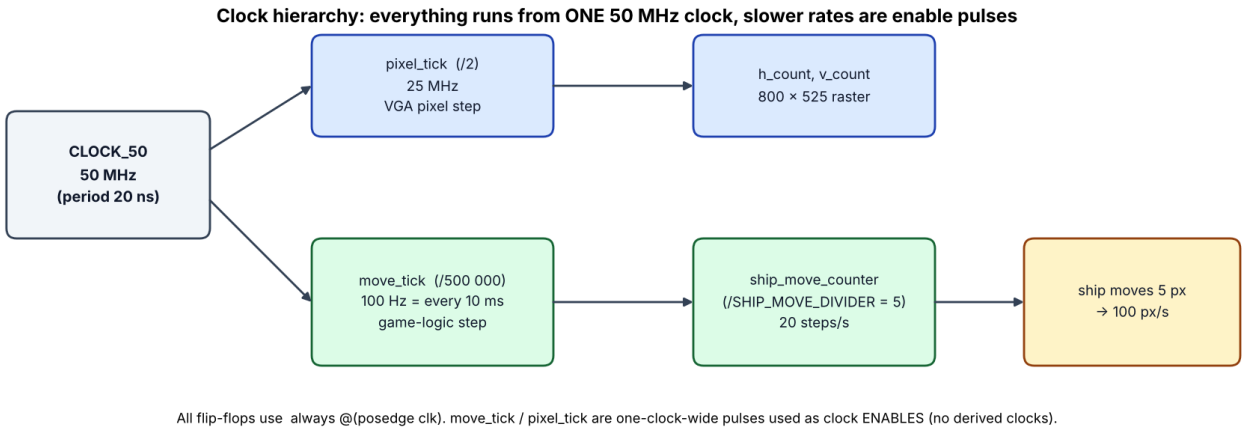


Figure 2. The two enable pulses derived from the 50 MHz clock.

4.1 move_tick (about 100 Hz)

All game motion (aliens, bullets, ship) advances only when **move_tick** is 1. The divider counts 0..499,999 and emits a one-clock pulse each time it wraps, i.e. every 10 ms. One “game tick” is therefore 10 ms; this is the unit used for all the speeds in this report.

src/clk_divider.v (src/clk_divider.v, lines 1-48, comment-only lines removed)

```
`timescale 1ns / 1ps
module clk_divider #(
    parameter integer DIVISOR = 500000
) (
    input  wire clk,
    input  wire not_reset,  // '0' = reset
    output reg  tick        // 1-cycle pulse at the divided rate
);
    reg [31:0] counter = 32'd0;
    always @(posedge clk) begin
        if (!not_reset) begin
            counter <= 32'd0;
            tick    <= 1'b0;
        end else if (counter == DIVISOR - 1) begin
            counter <= 32'd0;
            tick    <= 1'b1;
        end else begin
            counter <= counter + 32'd1;
            tick    <= 1'b0;
        end
    end
endmodule
```

4.2 pixel_tick (25 MHz)

A VGA 640x480 picture needs a pixel clock of about 25 MHz. **vga_controller** makes a toggle flip-flop (**pixel_tick**) that is 1 on every second 50 MHz clock; the raster counters advance only on those clocks.

5. VGA controller

A VGA monitor draws the picture line by line. The controller keeps two counters: **h_count** (0..799, the position along the line, counted in pixels) and **v_count** (0..524, the line number). Only the first 640 x 480 positions are visible; the rest is the blanking interval, which contains the sync pulses that tell the monitor when a line or a frame begins.

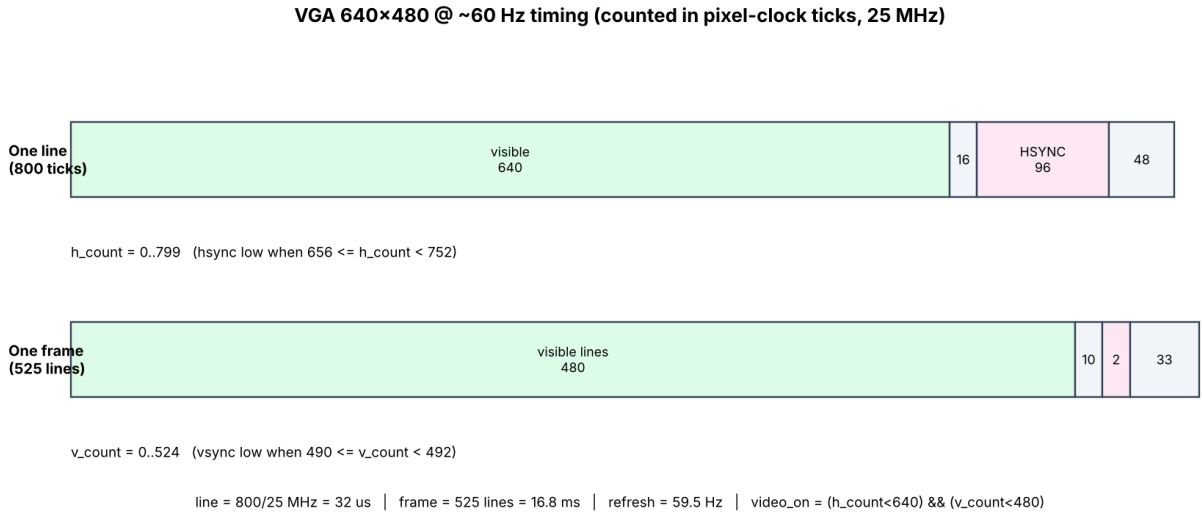


Figure 3. Horizontal and vertical timing. FP = front porch, BP = back porch.

src/vga_controller.v (src/vga_controller.v, lines 27-41, 46-53, 58-73, 78-87, comment-only lines removed)

```
localparam H_DISPLAY = 640;
localparam H_FRONT   = 16;
localparam H_SYNC_W  = 96;
localparam H_BACK    = 48;
localparam H_MAX     = H_DISPLAY + H_FRONT + H_SYNC_W + H_BACK - 1; // 799
localparam V_DISPLAY = 480;
localparam V_FRONT   = 10;
localparam V_SYNC_W  = 2;
localparam V_BACK    = 33;
localparam V_MAX     = V_DISPLAY + V_FRONT + V_SYNC_W + V_BACK - 1; // 524
reg        pixel_tick = 1'b0;   // /2 divider off the 50 MHz clk
reg [9:0]  h_count    = 10'd0;
reg [9:0]  v_count    = 10'd0;
...
always @(posedge clk) begin
    if (!not_reset)
        pixel_tick <= 1'b0;
```

```
    else
        pixel_tick <= ~pixel_tick;
end
assign p_tick = pixel_tick;
...
always @(posedge clk) begin
    if (!not_reset) begin
        h_count <= 10'd0;
        v_count <= 10'd0;
    end else if (pixel_tick) begin
        if (h_count == H_MAX) begin
            h_count <= 10'd0;
            if (v_count == V_MAX)
                v_count <= 10'd0;
            else
                v_count <= v_count + 10'd1;
        end else begin
            h_count <= h_count + 10'd1;
        end
    end
end
...
assign hsync = ~((h_count >= (H_DISPLAY + H_FRONT)) &&
                 (h_count <  (H_DISPLAY + H_FRONT + H_SYNC_W)));
assign vsync = ~((v_count >= (V_DISPLAY + V_FRONT)) &&
                 (v_count <  (V_DISPLAY + V_FRONT + V_SYNC_W)));
assign video_on = (h_count < H_DISPLAY) && (v_count < V_DISPLAY);
assign pixel_x = h_count;
assign pixel_y = v_count;
```

Key points: **hsync** and **vsync** are *active low* (they are 0 during the pulse). **video_on** is 1 only inside the visible 640x480 area; outside it the colour output is forced to black (the monitor needs black during blanking to calibrate itself). **pixel_x** and **pixel_y** are simply the counters: they tell the graphics module which pixel is being painted right now. Because the pixel clock is exactly 25 MHz (the standard is 25.175 MHz) the refresh rate is 59.5 Hz, which monitors accept.

6. The graphics module: geometry, colours and sprites

graphics is the heart of the design. Its inputs are the clock, reset, the raster position (px_x, px_y, video_on), the slow tick, the three conditioned buttons and the mode; its outputs are the 3-bit colour **rgb** and the score, level, lives and status flags. Screen coordinates start at the top-left corner (0,0); x grows to the right and y grows *downwards*.

6.1 Colours

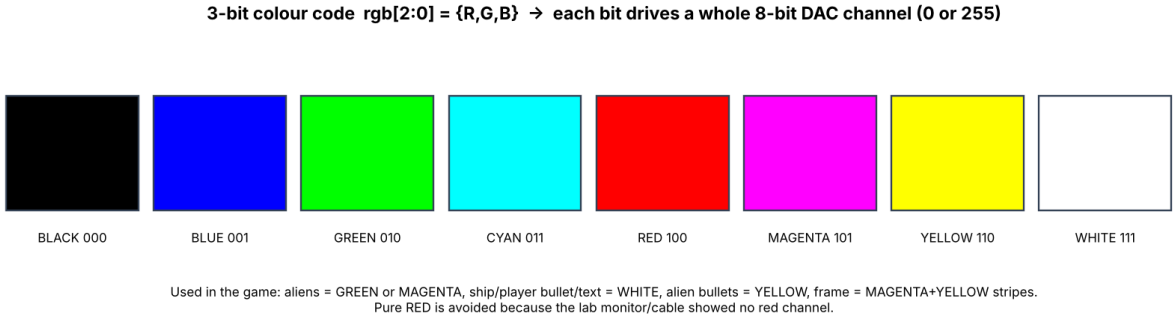


Figure 4. The 3-bit colour code. Each bit switches one colour channel fully on or off.

src/graphics.v (src/graphics.v, lines 133-140, 191-198)

```
localparam [2:0] COLOR_BLACK   = 3'b000;
localparam [2:0] COLOR_BLUE    = 3'b001;
localparam [2:0] COLOR_GREEN   = 3'b010;
localparam [2:0] COLOR_CYAN    = 3'b011;
localparam [2:0] COLOR_RED     = 3'b100;
localparam [2:0] COLOR_MAGENTA = 3'b101;
localparam [2:0] COLOR_YELLOW  = 3'b110;
localparam [2:0] COLOR_WHITE   = 3'b111;
...
// NOTE: no pure red is used for anything that must be seen. On the lab board's
// monitor/cable the red channel did not reach the screen (red parts came out
// black, yellow came out green, white came out cyan), so every in-game colour
// contains green and/or blue. Change these constants if your red channel works.
localparam [2:0] ALIEN_COLOR_A      = COLOR_GREEN;   // aliens, colour set A (random per alien)
localparam [2:0] ALIEN_COLOR_B      = COLOR_MAGENTA; // aliens, colour set B (random per alien)
localparam [2:0] BULLET_COLOR       = COLOR_WHITE;   // player bullet
localparam [2:0] ALIEN_BULLET_COLOR = COLOR_YELLOW;  // alien bullets
```

6.2 Sprite geometry

The ship, aliens and bullets are small bitmaps. Each bitmap pixel is drawn as a 2 x 2 block of screen pixels (**SPR_SCALE** = 2), which makes the game easier to see than the original paper's tiny sprites while still leaving room for 21 aliens. Resulting sizes: alien 32 x 18 px, ship 40 x 16 px, bullets 4 x 14 px.

src/graphics.v (src/graphics.v, lines 154-163, 168-176, 182-190)

```
localparam STAR_ROWS = 9,  STAR_COLS = 16;   // alien sprite box (name kept for the case-statement lookup below)
localparam SHIP_ROWS = 8,  SHIP_COLS = 20;     // ship sprite bitmap size

// On screen every sprite pixel is drawn SPR_SCALE x SPR_SCALE screen pixels,
// so the in-game sizes are:  alien 32 x 18, ship 40 x 16, bullets 4 x 14.
localparam SPR_SCALE = 2;
localparam ALIEN_W   = STAR_COLS * SPR_SCALE;   // 32
localparam ALIEN_H   = STAR_ROWS * SPR_SCALE;   // 18
localparam SHIP_W    = SHIP_COLS * SPR_SCALE;   // 40
localparam SHIP_H    = SHIP_ROWS * SPR_SCALE;   // 16
...
localparam SHIP_Y      = 440;
localparam SHIP_X_INIT = (SCREEN_WIDTH / 2) - (SHIP_W / 2);
// The frame (border) is drawn 8..11 px in from every edge, so the playfield
// is x 12..627, y 12..467. The ship stays 4 px inside it; alien bullets are
// removed just before they reach the bottom frame (FLOOR_Y).
localparam SHIP_X_MIN  = 16;
localparam SHIP_X_MAX  = 624 - SHIP_W;
localparam FLOOR_Y     = 468;
localparam SHIP_STEP   = 5;   // matches the paper's "+5 / -5" step
...
localparam ALIEN_COLS    = 7;
localparam ALIEN_Y0      = 60;
localparam ALIEN_SPACING = 48;
localparam ALIEN_X0      = (SCREEN_WIDTH - ((ALIEN_COLS - 1) * ALIEN_SPACING + ALIEN_W)) / 2;
localparam ROW_SPACING   = 26;                      // vertical gap between alien rows
localparam TOTAL_ALIENS  = ALIEN_COLS * ALIEN_ROWS;

localparam BULLET_W = 4;
localparam BULLET_H = 14;
```

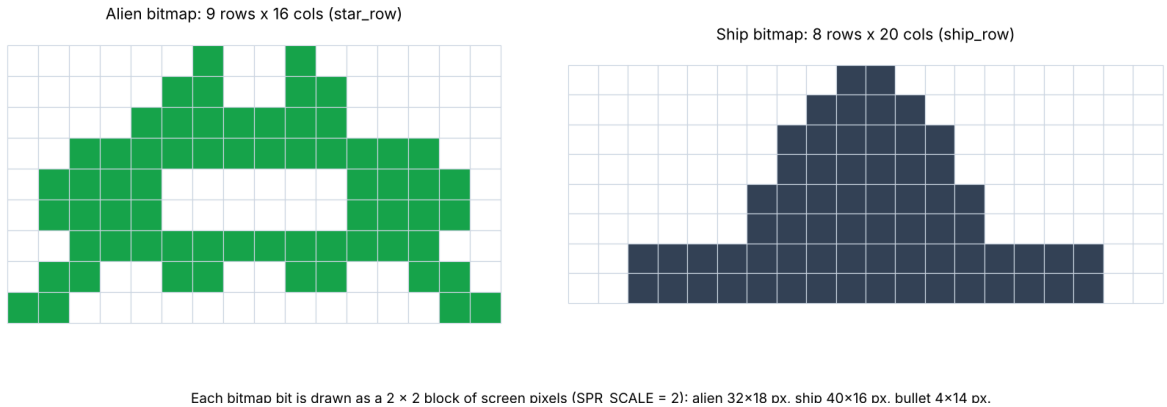


Figure 5. The sprite bitmaps. A 1 in the bitmap is a lit pixel.

A sprite is stored as a Verilog function that returns one row of bits for a given row number, written as a **case** statement; the MSB is the leftmost pixel. The same function is used by every alien, so one bitmap serves all 21 of them.

star_row: the alien bitmap (src/graphics.v, lines 220-236)

```
function [STAR_COLS-1:0] star_row;
    input [3:0] r;
    begin
        case (r)
            4'd0: star_row = 16'b0000001001000000;
            4'd1: star_row = 16'b0000011001100000;
            4'd2: star_row = 16'b0000111111100000;
            4'd3: star_row = 16'b0011111111111100;
            4'd4: star_row = 16'b0111100000011110;
            4'd5: star_row = 16'b0111100000011110;
            4'd6: star_row = 16'b0011111111111100;
            4'd7: star_row = 16'b0110011001100110;
            4'd8: star_row = 16'b1100000000000011;
            default: star_row = 16'b0;
        endcase
    end
endfunction
```

6.3 Play-area limits

The border frame occupies x = 8..11 and 628..631, y = 8..11 and 468..471, so the usable play area is x 12..627, y 12..467. The ship stays 4 px inside the frame (x from 16 to 584, because it is 40 px wide), sits on row **SHIP_Y** = 440, and alien bullets disappear at **FLOOR_Y** = 468, just before the bottom frame.

7. Randomness: the LFSRs

Digital logic is deterministic, so “random” behaviour is produced with a **Linear Feedback Shift Register (LFSR)**: a shift register whose new input bit is the XOR of a few fixed “tap” bits. With well-chosen taps a 32-bit LFSR visits all 2^{32} - 1 non-zero states before repeating, in an order that looks random.

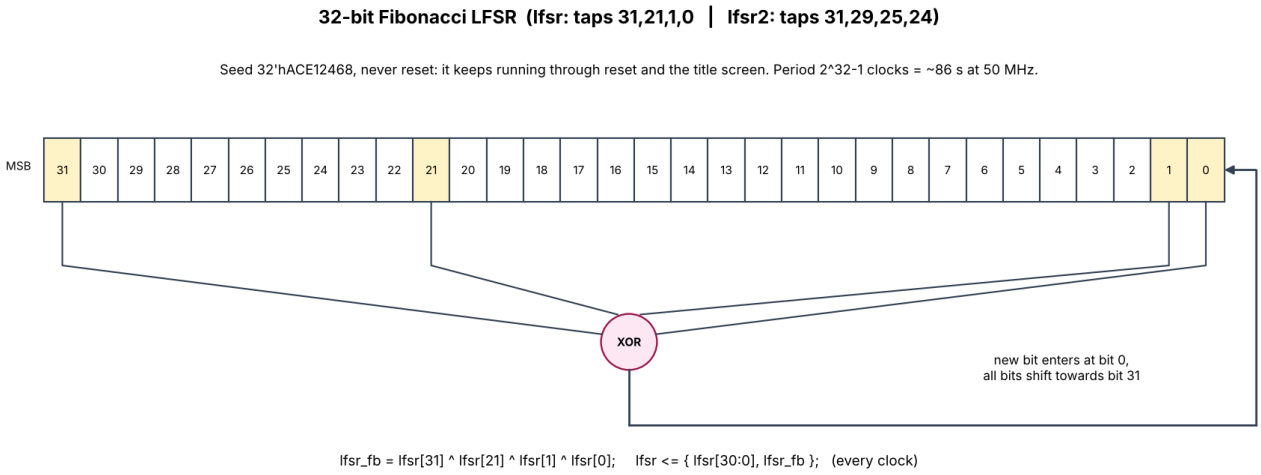


Figure 6. The 32-bit LFSR. A second one, lfsr2, uses a different polynomial (taps 31, 29, 25, 24) and seed.

src/graphics.v (src/graphics.v, lines 330-335, 340-340, 348-351)

```
reg [31:0] lfsr = 32'hACE12468;   // any non-zero value
wire       lfsr_fb = lfsr[31] ^ lfsr[21] ^ lfsr[1] ^ lfsr[0];

always @(posedge clk) begin
    lfsr <= {lfsr[30:0], lfsr_fb};
end
...
reg [31:0] lfsr2 = 32'h5EED1234;
...
wire       lfsr2_fb = lfsr2[31] ^ lfsr2[29] ^ lfsr2[25] ^ lfsr2[24];
always @(posedge clk) begin
    lfsr2 <= {lfsr2[30:0], lfsr2_fb};
end
```

Why free-running and never reset? The LFSR keeps shifting through reset and through the whole title screen (about 86 s per full cycle). The moment the player presses a key therefore decides where in the sequence the game starts, so each game is different. The title-screen block copies LFSR values into the game's countdowns and the colour register while it waits, and those copies freeze at the key press.

Slices. Different decisions use different groups of bits of the same register so that they are not accidentally correlated:

Decision	LFSR bits used	Result
Descent wait (rigid block)	r[5:0] (EASY), r[3:0] (MEDIUM)	wait = DESC_MIN + value ticks
Descent step	r[7:6]	1, 2, 3 or 2 px (average 2)
Sideways wait	r[12:8] (EASY), r[10:8] (MEDIUM)	wait = HORIZ_MIN + value ticks
Random direction reversal	r[15:13] == 0 and r[20:16] == 0	1 chance in 256 per sideways step
Alien fire wait	r[23:16] (EASY), r[21:16] (MEDIUM, HARD)	wait = FIRE_MIN + value ticks
Which alien fires	r[31:24] mod 21 (shooter A); lfsr2[31:24] mod 21 (B)	index 0..20
Alien colours	lfsr2[20:0] XOR lfsr[31:11]	one bit per alien (acol)

src/graphics.v (src/graphics.v, lines 364-371, 394-402)

```
function [15:0] desc_delay;
    input        hd;
    input [31:0] r;
    begin
        if (hd) desc_delay = MED_DESC_MIN + r[3:0];
        else    desc_delay = EASY_DESC_MIN + r[5:0];
    end
endfunction
```

```
...
function [15:0] fire_delay;
    input [1:0]  m;                   // 0 EASY, 1 MEDIUM, 2 HARD
    input [31:0] r;
    begin
        if (m[1])      fire_delay = HARD_FIRE_MIN + r[21:16];
        else if (m[0]) fire_delay = MED_FIRE_MIN  + r[21:16];
        else           fire_delay = EASY_FIRE_MIN + r[23:16];
    end
endfunction
```

The helper functions read the LFSR combinationally; because they are only used when a countdown reaches zero (every few hundred milliseconds), the LFSR is effectively sampled at unpredictable instants.

8. The game state machine

The register **state** holds one of four values. Everything in the game block is guarded by it: nothing moves on the title screen, the whole game is frozen on the OVER and WON screens, and the pixel mux also selects which picture to draw according to **state**.

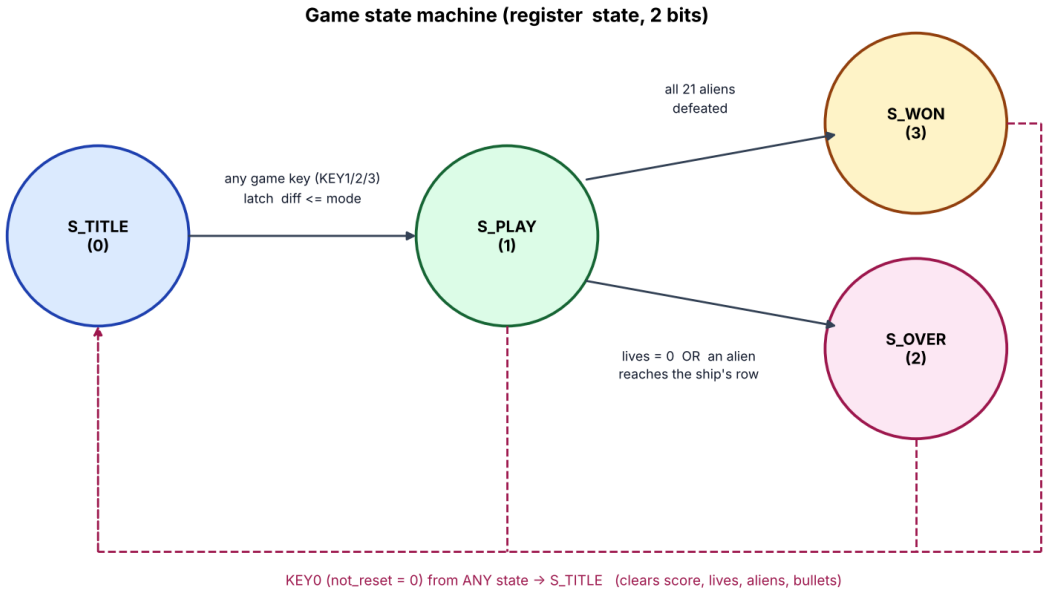


Figure 7. The four game states and their transitions. KEY0 can interrupt any state.

src/graphics.v (src/graphics.v, lines 146-149)

```
localparam [1:0] S_TITLE = 2'd0;
localparam [1:0] S_PLAY  = 2'd1;
localparam [1:0] S_OVER  = 2'd2;
localparam [1:0] S_WON   = 2'd3;
```

8.1 Title state: latching the mode

While in S_TITLE the block keeps copying the live mode switches into **diff** (the difficulty in force) and the random colour bits into **acol**, and pre-loads the three random countdowns. When any of the three game keys is pressed the state becomes S_PLAY and the values copied on that last clock stay frozen: flipping the switches later has no effect on the running game.

src/graphics.v (src/graphics.v, lines 264-268, 633-654)

```
wire [1:0] mode_n   = mode[1] ? 2'd2 : {1'b0, mode[0]};   // 3 is treated as HARD
reg [1:0] diff      = 2'd0;        // difficulty in force this game (latched from mode at start): 0 EASY, 1 MEDIUM, 2 HARD
wire      indep     = diff[1];     // HARD: every alien moves independently
wire      med       = diff[0];     // MEDIUM flag for the rigid-formation timing functions
wire      any_key   = nes_left | nes_right | shoot;
...
    end else if (state == S_TITLE) begin
        //---------------------------------------------------------
        // TITLE: nothing about the game itself moves. Keep sampling the
        // mode switches (what they read at the instant a key is pressed
        // is what gets latched into `diff`), keep the three random
        // countdowns pre-loaded from the live mode, and wait for ANY of
        // the three game keys. On the start press, nes_a/nes_b are
        // preset to 1 so that if the key that started the game is the
        // shoot key it is NOT also seen as a fresh rising edge (i.e. no
        // free bullet).
        //---------------------------------------------------------
        diff              <= mode_n;
        acol              <= lfsr2[TOTAL_ALIENS-1:0] ^ lfsr[31 -: TOTAL_ALIENS];
        descend_counter   <= desc_delay(mode_n[0], lfsr);
        horiz_counter     <= horiz_delay(mode_n[0], lfsr);
```

```
        alien_fire_counter <= fire_delay(mode_n, lfsr);
        alien_fire_counter_b <= fire_delay(mode_n, lfsr2);
        if (any_key) begin
            state <= S_PLAY;
            nes_a <= 1'b1;
            nes_b <= 1'b1;
        end
```

nes_a/nes_b are the two-flop edge detector for the shoot key (Section 11). They are preset to 1 at start so that, if the key that started the game was the shoot key, the game does not treat it as a fresh press and fire a stray bullet.

8.2 Difficulty flags

diff is a 2-bit code: 0 = EASY, 1 = MEDIUM, 2 = HARD. Two single-bit views are used all over the design: **indep = diff[1]** (HARD: every alien moves independently) and **med = diff[0]** (selects the MEDIUM timing constants in the rigid-block functions).

9. EASY and MEDIUM: the rigid random formation

In these modes the 21 aliens form a 7-column by 3-row block. The block's position is stored in just three registers: **x_offset** (sideways position, 0..288), **descent_offset** (how far it has sunk) and the direction flag. Each alien's coordinates are computed from those plus its row and column, so the aliens always move together as one rigid group.



Figure 8. Geometry of the rigid formation. The ghost blocks show the two extreme sideways positions.

The motion is not regular but random: two independent countdowns, one for descent and one for sideways movement, run once per move_tick. When a countdown reaches 0 the step is taken and the countdown is reloaded with *minimum* + *random* ticks. The direction reverses at the limits (x_offset = 0 or 288) and also at random once in about 256 steps, so the block wanders instead of sweeping like a clock.

src/graphics.v (src/graphics.v, lines 711-718, 727-748)

```
if (move_tick && !indep) begin
    if (descend_counter == 16'd0) begin
        descend_counter <= desc_delay(med, lfsr);
        descent_offset  <= descent_offset + desc_step(lfsr[7:6]);
    end else begin
        descend_counter <= descend_counter - 16'd1;
    end
end

if (move_tick && !indep) begin
    if (horiz_counter == 16'd0) begin
        horiz_counter <= horiz_delay(med, lfsr);
        if (move_dir_right) begin
            if (x_offset >= (2 * MAX_SWEEP))
                move_dir_right <= 1'b0;                 // hit the right limit: bounce
            else if ((lfsr[15:13] == 3'd0) && (lfsr[20:16] == 5'd0))
                move_dir_right <= 1'b0;                 // random reversal (1 in 256)
            else
                x_offset <= x_offset + 9'd1;
        end else begin
            if (x_offset == 9'd0)
                move_dir_right <= 1'b1;                 // hit the left limit: bounce
            else if ((lfsr[15:13] == 3'd0) && (lfsr[20:16] == 5'd0))
                move_dir_right <= 1'b1;                 // random reversal (1 in 256)
```

```
            else
                x_offset <= x_offset - 9'd1;
        end
    end else begin
        horiz_counter <= horiz_counter - 16'd1;
    end
end
```

The values of the minimum waits and the resulting speeds for each mode are listed in Section 16.

10. HARD mode: 21 independent agents

In HARD mode there is no formation. A **generate** loop creates 21 copies of a small state machine, one per alien. Each copy has its own 16-bit LFSR (**rnd**, a different seed per alien), its own position (x, y), its own direction and speed classes, and its own countdown timers. A copy reads only the shared clock, tick and FSM state and its own bit of **defeated**; it never reads another alien's registers, so the aliens cannot move as a block even by accident.

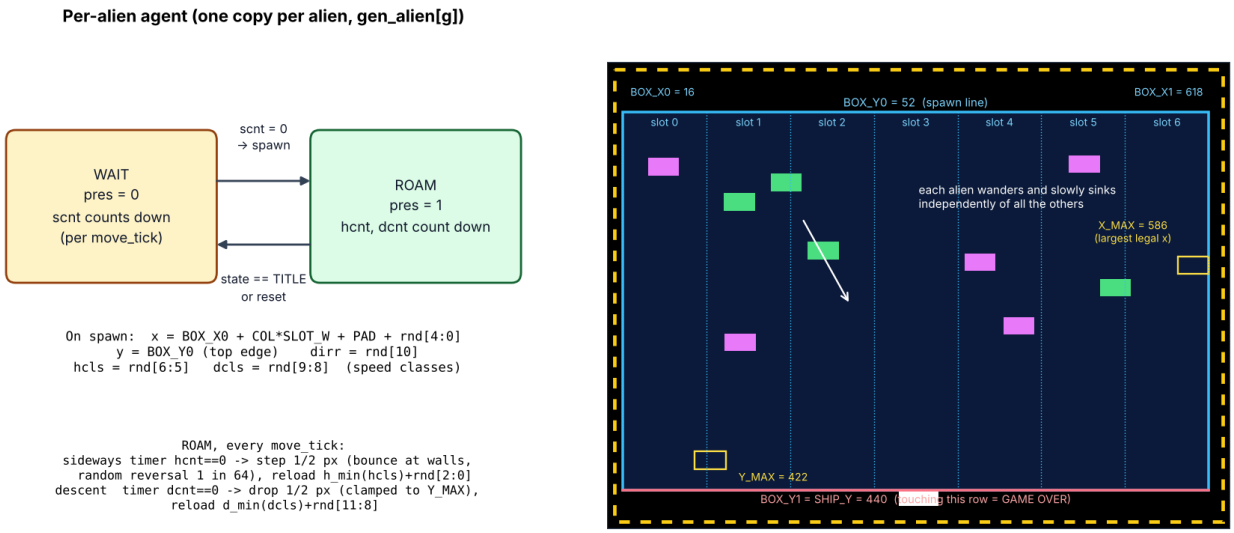


Figure 9. Left: the per-alien WAIT/ROAM machine. Right: the play rectangle (x 16..618, y 52..440) with seven spawn slots.

gen_alien: per-alien registers (src/graphics.v, lines 472-495)

```
genvar g;
generate
    for (g = 0; g < TOTAL_ALIENS; g = g + 1) begin : gen_alien
        localparam integer COL = g % ALIEN_COLS;
        localparam integer ROW = g / ALIEN_COLS;
        localparam [15:0]  SEED = 16'hACE1 ^ (g * 16'h9E37);

        reg [15:0] rnd  = SEED;
        wire       rfb  = rnd[15] ^ rnd[14] ^ rnd[12] ^ rnd[3];
        reg        pres = 1'b0;
        reg [9:0]  x    = 10'd0;
        reg [9:0]  y    = 10'd0;
        reg        dirr = 1'b1;              // 1 = moving right
        reg [1:0]  hcls = 2'd1;              // sideways speed class
        reg [1:0]  dcls = 2'd1;              // descent speed class
        reg [9:0]  hcnt = 10'd0;             // sideways countdown
        reg [9:0]  dcnt = 10'd0;             // descent countdown
        reg [9:0]  scnt = 10'd0;             // spawn countdown (WAIT)

        wire [1:0] hstep = rnd[3]  ? 2'd2 : 2'd1;   // this alien's next sideways step size
        wire [1:0] dstep = rnd[12] ? 2'd2 : 2'd1;   // this alien's next drop size
        wire [9:0] x_fwd = x + hstep;
        wire [9:0] y_fwd = y + dstep;
```

gen_alien: the state machine (src/graphics.v, lines 496-542)

```
always @(posedge clk) begin
    rnd <= {rnd[14:0], rfb};

    if (!not_reset || state == S_TITLE) begin
        pres <= 1'b0;
        scnt <= HARD_SPAWN_GAP * ROW + rnd[6:0];
    end else if (indep && move_tick && state == S_PLAY) begin
        if (!pres) begin
            //---------------- WAIT -> spawn on the top edge
            if (scnt == 10'd0) begin
                pres <= 1'b1;
                x    <= BOX_X0 + COL * SLOT_W + SPAWN_PAD + rnd[4:0];
                y    <= BOX_Y0;
                dirr <= rnd[10];
                hcls <= rnd[6:5];
                dcls <= rnd[9:8];
```

```
                hcnt <= {6'd0, rnd[3:0]};
                dcnt <= {6'd0, rnd[15:12]};
            end else begin
                scnt <= scnt - 10'd1;
            end
        end else if (!defeated[g]) begin
            //---------------- ROAM: sideways
            if (hcnt == 10'd0) begin
                hcnt <= h_min(hcls) + rnd[2:0];
                if (dirr) begin
                    if (x_fwd > X_MAX)        dirr <= 1'b0;      // right wall: bounce
                    else if ((rnd[6:4] == 3'd0) && (rnd[9:7] == 3'd0)) dirr <= 1'b0;   // random reversal (1 in 64)
                    else                       x <= x_fwd;
                end else begin
                    if (x < BOX_X0 + hstep)    dirr <= 1'b1;      // left wall: bounce
                    else if ((rnd[6:4] == 3'd0) && (rnd[9:7] == 3'd0)) dirr <= 1'b1;   // random reversal (1 in 64)
                    else                       x <= x - hstep;
                end
            end else begin
                hcnt <= hcnt - 10'd1;
            end
            //---------------- ROAM: descent
            if (dcnt == 10'd0) begin
                dcnt <= d_min(dcls) + rnd[11:8];
                y    <= (y_fwd > Y_MAX) ? Y_MAX : y_fwd;
            end else begin
                dcnt <= dcnt - 10'd1;
            end
        end
    end
end
```

WAIT. During the title screen (and reset) each alien is absent and its spawn timer **scnt** is loaded with HARD_SPAWN_GAP x ROW + a random 0..127 ticks, so row 0 appears first, then row 1, then row 2. In play the timer counts down once per move_tick; at 0 the alien appears on the top edge (y = BOX_Y0 = 52) at a random x inside its own column slot, with a random direction and random speed classes.

ROAM. Two countdowns run every move_tick. The sideways timer **hcnt** moves the alien 1 or 2 px left/right (bouncing at the rectangle walls and reversing at random 1 time in 64); the descent timer **dcnt** drops it 1 or 2 px. The wait after each step is a class minimum (**h_min**, **d_min**) plus a random value, so fast and slow aliens coexist. The position is always clamped inside the rectangle, so aliens never leave it: x stays within [16, 586] and y within [52, 422].

Which position is used? The rest of the design never looks at where an alien “really” is; it reads two flat vectors, **apx_flat** and **apy_flat**, where slice k holds alien k's x and y. The generate block fills them with the alien's own registers in HARD mode and with the rigid-formation formula in EASY/MEDIUM. Drawing, collisions, alien fire and the touch check all use these vectors, so they work unchanged for both kinds of motion.

src/graphics.v (src/graphics.v, lines 438-443, 544-549)

```
wire [10*TOTAL_ALIENS-1:0] hx_flat, hy_flat, apx_flat, apy_flat;
wire [TOTAL_ALIENS-1:0]    hp_flat;
wire [TOTAL_ALIENS-1:0]    present = indep ? hp_flat : {TOTAL_ALIENS{1'b1}};
wire [TOTAL_ALIENS-1:0]    alive   = present & ~defeated;
// y at which the player's bullet gives up (the top of the alien area)
wire [9:0] top_y = indep ? BOX_Y0 : (ALIEN_Y0 + descent_offset);
...
        assign hx_flat[10*g +: 10] = x;
        assign hy_flat[10*g +: 10] = y;
        assign hp_flat[g]          = pres;
        assign apx_flat[10*g +: 10] = indep ? x : (ALIEN_X0 + COL * ALIEN_SPACING + x_offset - MAX_SWEEP);
        assign apy_flat[10*g +: 10] = indep ? y : (ALIEN_Y0 + ROW * ROW_SPACING + descent_offset);
    end
```

11. Ship, player bullet and collisions

11.1 Ship movement

move_tick (100 Hz) is far too fast to step the ship on every tick, since even a quick button tap lasts 10-20 ticks. A small counter divides it by SHIP_MOVE_DIVIDER = 5, so the ship moves 5 px per step, 20 steps per second (100 px/s) while a key is held. The ship is clamped between SHIP_X_MIN and SHIP_X_MAX and re-centred after a lost life.

src/graphics.v (src/graphics.v, lines 567-585)

```
always @(posedge clk) begin
    if (!not_reset) begin
        ship_x            <= SHIP_X_INIT;
        ship_move_counter <= 16'd0;
    end else if (life_lost_pending) begin
        ship_x            <= SHIP_X_INIT;
        ship_move_counter <= 16'd0;
    end else if ((state == S_PLAY) && move_tick) begin
        if (ship_move_counter == SHIP_MOVE_DIVIDER - 1) begin
            ship_move_counter <= 16'd0;
```

```
            if (nes_right && (ship_x + SHIP_STEP <= SHIP_X_MAX))
                ship_x <= ship_x + SHIP_STEP;
            else if (nes_left && (ship_x >= SHIP_X_MIN + SHIP_STEP))
                ship_x <= ship_x - SHIP_STEP;
        end else begin
            ship_move_counter <= ship_move_counter + 16'd1;
        end
    end
end
```

11.2 Shooting: edge detection

A bullet must be launched once per *press*, not once per clock while the key is held. The shoot input goes through two flip-flops, **nes_a** (now) and **nes_b** (one clock earlier). A rising edge is “now = 1 and before = 0”. Only one player bullet exists at a time: a new one cannot start while **bullet_active** is 1.

11.3 Bullet flight and hit test

The bullet starts at the middle of the ship's top edge and moves up 4 px per move_tick. On every tick, while it flies, it is tested against all 21 alien boxes using their *current* positions (essential in HARD mode, where aliens move). A hit needs the horizontal ranges to overlap and the bullet's y to fall inside the alien's box. A hit marks the alien **defeated** (it stops being drawn and can no longer fire), consumes the bullet and adds 2 to the score. A bullet that reaches the top of the alien area without a hit simply disappears.

src/graphics.v (src/graphics.v, lines 674-704)

```
if (nes_a && !nes_b && !bullet_active) begin
    bullet_active <= 1'b1;
    bullet_x <= ship_x + (SHIP_W / 2) - (BULLET_W / 2);
    bullet_y <= SHIP_Y - 1;
end else if (move_tick && bullet_active) begin
    for (k = 0; k < TOTAL_ALIENS; k = k + 1) begin
        cx = apx_flat[10*k +: 10];
        cy = apy_flat[10*k +: 10];
        if (!hit && alive[k] &&
            (bullet_x + BULLET_W >= cx) &&
            (bullet_x <= cx + ALIEN_W) &&
            (bullet_y >= cy) &&
            (bullet_y <= cy + ALIEN_H + BULLET_H)) begin
            defeated[k] <= 1'b1;
            hit = 1'b1;
        end
    end

    if (hit)
        bullet_active <= 1'b0;                          // consumed by a hit
    else if (bullet_y <= top_y)
        bullet_active <= 1'b0;                          // flew past every alien, missed
    else
        bullet_y <= bullet_y - BULLET_SPEED[9:0];       // keep travelling up
end

destruction <= hit;

// score_next <= score+2 when destruction='1' else score;
if (hit)
    score_reg <= score_reg + 8'd2;
```



Figure 10. Order of events inside the PLAY branch of the main game block. Because later assignments to the same register override earlier ones within one clock, the order matters: the touch check is last on purpose.

12. Alien return fire, lives and the end conditions

12.1 Two independent shooters

Each shooter has its own countdown, its own bullet and its own random pick, so two different aliens can have a bullet in flight at the same time. When the countdown reaches 0 it is reloaded (random wait). If that shooter's bullet is not already in flight, an alien index is picked from the LFSR; if that alien is not alive (already shot, or in HARD mode not yet spawned) the countdown is set to 0 so another alien is tried on the very next tick, which keeps the firing rate up as the aliens thin out. The bullet leaves from the centre of the bottom edge of the alien's *current* box.

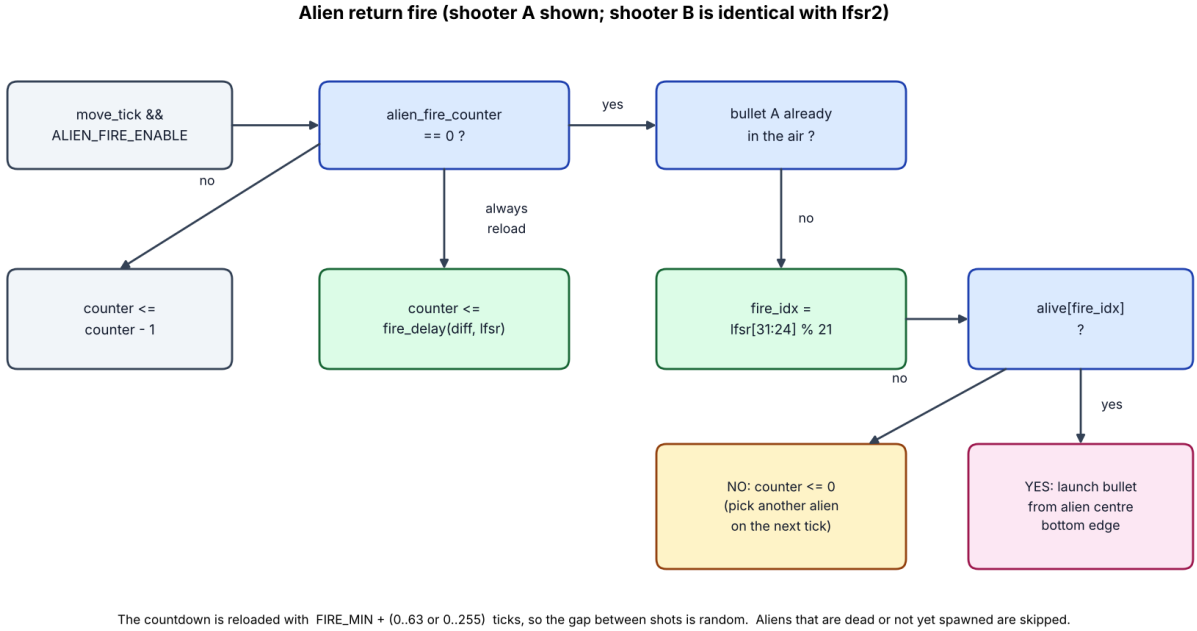


Figure 11. Alien fire logic for shooter A (shooter B is identical but uses lfsr2).

shooter A (src/graphics.v, lines 760-776)

```
if (move_tick && (ALIEN_FIRE_ENABLE != 0)) begin
    if (alien_fire_counter == 16'd0) begin
        alien_fire_counter <= fire_delay(diff, lfsr);
        if (!alien_bullet_active) begin
```

```
            fire_idx = lfsr[31:24] % TOTAL_ALIENS;
            if (!alive[fire_idx]) begin
                alien_fire_counter <= 16'd0;   // dead / absent pick: try again next tick
            end else begin
                alien_bullet_active <= 1'b1;
                alien_bullet_x <= apx_flat[10*fire_idx +: 10] + (ALIEN_W / 2) - (BULLET_W / 2);
                alien_bullet_y <= apy_flat[10*fire_idx +: 10] + ALIEN_H;
            end
        end
    end else begin
        alien_fire_counter <= alien_fire_counter - 16'd1;
    end
end
```

Shooter B is a copy of this block with its own registers (alien_fire_counter_b, alien_bullet_b_*) and fire_idx_b taken from lfsr2.

12.2 Alien bullets and lives

Alien bullets fall 4 px per tick in EASY and 6 px per tick in MEDIUM and HARD. If a bullet overlaps the ship it costs one life (**life_lost**); if it reaches FLOOR_Y it vanishes. Lives go 3, 2, 1, then 0 (dead). On a non-fatal hit the ship returns to the centre and all bullets are cleared, but the aliens stay exactly where they are.

src/graphics.v (src/graphics.v, lines 805-815, 849-860)

```
if (alien_bullet_active && move_tick) begin
    if ((alien_bullet_x + BULLET_W >= ship_x) && (alien_bullet_x <= ship_x + SHIP_W) &&
        (alien_bullet_y + BULLET_H >= SHIP_Y) && (alien_bullet_y <= SHIP_Y + SHIP_H)) begin
        alien_bullet_active <= 1'b0;
        life_lost = 1'b1;
    end else if (alien_bullet_y + BULLET_H >= FLOOR_Y) begin
        alien_bullet_active <= 1'b0;   // missed: gone just before the bottom frame
    end else begin
        alien_bullet_y <= alien_bullet_y + abullet_speed;
    end
end

if (life_lost) begin
    if (lives_reg <= 2'd1) begin
        lives_reg <= 2'd0;
        state     <= S_OVER;
    end else begin
        lives_reg           <= lives_reg - 2'd1;
        life_lost_pending   <= 1'b1;
        bullet_active       <= 1'b0;
        alien_bullet_active <= 1'b0;
        alien_bullet_b_active <= 1'b0;
    end
end
```

12.3 Winning and losing

Win: defeated has one bit per alien; **&defeated;** (reduction AND) is 1 only when all 21 bits are set, and then the FSM goes to S_WON. (The extra **flip_counter** test is a leftover of the paper's "counter(0)" trick for incrementing the level exactly once.) **Lose:** either the lives reach 0, or *any* live alien has come down to the ship's row. The touch test is placed last so it wins over everything else that happens in the same clock.

src/graphics.v (src/graphics.v, lines 831-842, 862-875)

```
// restart_next <= '1' when all TOTAL_ALIENS defeated else '0';
// (paper: defeated1='1' and defeated2='1' and defeated3='1')
restart_next <= &defeated;
flip_counter <= ~flip_counter;

// level_next <= level+1 when counter(0)='0' and all defeated
// else level. Clearing every alien now ENDS the game: the FSM
// goes to WON (the "GAME WON" screen) instead of respawning.
if (!flip_counter && &defeated) begin
    level_reg <= level_reg + 2'd1;
    state     <= S_WON;
end

// ANY live alien that has come down to the ship's row (the
// bottom of the play area) is game over immediately, however
// many lives are left. Placed LAST so it wins over everything
// above that lands on the same cycle.
touch = 1'b0;
for (k = 0; k < TOTAL_ALIENS; k = k + 1) begin
    cy = apy_flat[10*k +: 10];
    if (alive[k] && (cy + ALIEN_H >= SHIP_Y))
        touch = 1'b1;
end
if (touch) begin
    lives_reg <= 2'd0;
    state     <= S_OVER;
end
```

Housekeeping note. The registers **restart_next** (write-only) and **destruction** (drives an unconnected debug output) are leftovers of the paper's equations; the synthesiser removes logic that has no effect, so they cost nothing in hardware.

13. Score, HUD text and the 7-segment display

The score rises by 2 per alien shot (maximum 42 with 21 aliens). It is stored in binary in **score_reg**. For the on-screen text it is split into decimal digits with divisions by constants; for the board it is sent as two hex digits to HEX0/HEX1 through **hex7seg**, and the level goes to HEX2.

src/graphics.v (src/graphics.v, lines 1071-1073)

```
wire [3:0] score_d2 = score_reg / 8'd100;
wire [3:0] score_d1 = (score_reg / 8'd10) % 8'd10;
wire [3:0] score_d0 = score_reg % 8'd10;
```

hex7seg: first entries of the lookup table (src/hex7seg.v, lines 23-31)

```
always @(*) begin
    case (hex)
        4'h0: seg = 7'b1000000;
        4'h1: seg = 7'b1111001;
        4'h2: seg = 7'b0100100;
        4'h3: seg = 7'b0110000;
        4'h4: seg = 7'b0011001;
        4'h5: seg = 7'b0010010;
        4'h6: seg = 7'b0000010;
```

13.1 How text is drawn without any memory

There is no frame buffer. Text is produced the same way as everything else: per pixel. A 5 x 7 font is stored as 35-bit constants (7 rows of 5 bits) in the function **font_row**. For a pixel inside a text area the logic computes (1) which character cell it is in, (2) which glyph that cell holds, (3) which row/column of the glyph the pixel is, and (4) whether that glyph bit is 1. A character is drawn at a scale of 2, 3 or 6 screen pixels per font pixel depending on where it is used (HUD, prompts, title).

font glyphs: digit 0, the letter A, and the row selection (src/graphics.v, lines 931-932, 950-950, 963-963)

```
    CH_0: glyph = 35'b01110_10001_10011_10101_11001_10001_01110;
    CH_1: glyph = 35'b00100_01100_00100_00100_00100_00100_01110;

    CH_A: glyph = 35'b01110_10001_10001_11111_10001_10001_10001;

font_row = glyph[34 - row*5 -: 5];
```

HUD: the SCORE line (src/graphics.v, lines 1260-1281)

```
if ((px_x >= HUD_X0) && (px_x < HUD_X0 + HUD_CHARS*HUD_PITCH) &&
    (px_y >= HUD_Y0) && (px_y < HUD_Y0 + HUD_CHAR_H)) begin
    hud_char_i = (px_x - HUD_X0) / HUD_PITCH;
    hud_sub_x  = (px_x - HUD_X0) % HUD_PITCH;
    hud_sub_y  = px_y - HUD_Y0;
    if (hud_sub_x < HUD_CHAR_W) begin
        case (hud_char_i)
            0: hud_ch = CH_S;
            1: hud_ch = CH_C;
            2: hud_ch = CH_O;
            3: hud_ch = CH_R;
            4: hud_ch = CH_E;
            5: hud_ch = CH_SP;
            6: hud_ch = CH_0 + score_d2;
            7: hud_ch = CH_0 + score_d1;
            8: hud_ch = CH_0 + score_d0;
            default: hud_ch = CH_SP;
        endcase
        font_bits = font_row(hud_ch, hud_sub_y / HUD_SCALE);
        if (font_bits[FONT_W-1 - (hud_sub_x / HUD_SCALE)])
            hud_hit = 1'b1;
    end
```

14. Rendering: the pixel colour mux

This is the part that finally produces what you see. It is one combinational block (**always @***) that, for the current pixel (px_x, px_y), evaluates in parallel whether the pixel lies on an alien, on the ship, on a bullet, on HUD text, on the frame, on a star, and so on, and then chooses one colour using a priority chain (first match wins).

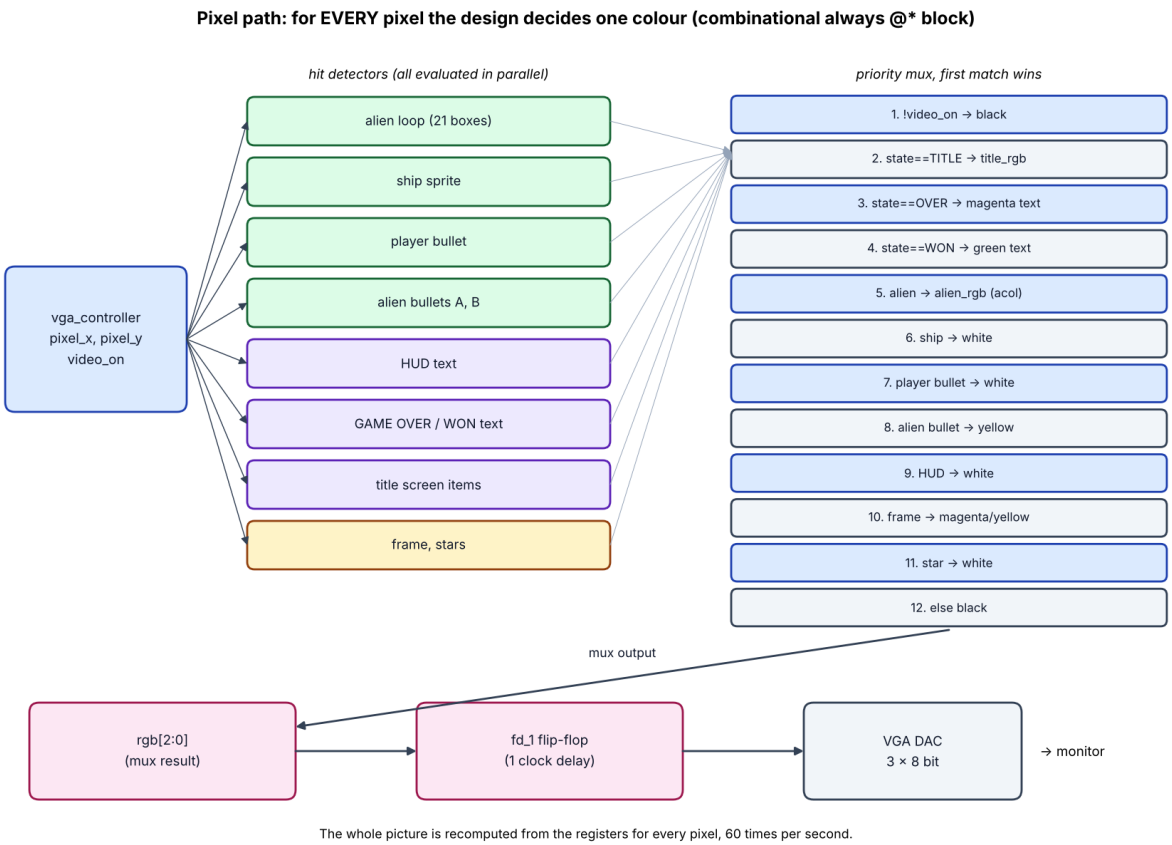


Figure 12. Pixel path: parallel hit detectors feed a priority mux whose result goes through the output flip-flop.

14.1 Aliens

A loop over the 21 aliens checks whether the current pixel is inside alien m's 32 x 18 box; if so, the sprite row is looked up (row = (px_y - ay) / 2) and the bit for the column ((px_x - ax) / 2) decides whether this pixel is lit. The alien's colour comes from its bit in **acol**: 0 = green, 1 = magenta. Because acol is random per alien and frozen for the game, the two colours are mixed within every row, and in HARD mode each alien keeps its colour wherever it wanders.

src/graphics.v (src/graphics.v, lines 1217-1237)

```
alien_hit = 1'b0;
alien_rgb = ALIEN_COLOR_A;
for (m = 0; m < TOTAL_ALIENS; m = m + 1) begin
    if (!alien_hit && alive[m]) begin
        ax = apx_flat[10*m +: 10];
        ay = apy_flat[10*m +: 10];
        if ((px_x >= ax) && (px_x < ax + ALIEN_W) &&
            (px_y >= ay) && (px_y < ay + ALIEN_H)) begin
            alien_bits = star_row((px_y - ay) / SPR_SCALE);
            if (alien_bits[STAR_COLS-1 - (px_x - ax) / SPR_SCALE]) begin
                alien_hit = 1'b1;
                // two colour sets, assigned randomly per alien (acol)
                alien_rgb = acol[m] ? ALIEN_COLOR_B : ALIEN_COLOR_A;
            end
        end
    end
end

ship_hit = (px_x >= ship_x) && (px_x < ship_x + SHIP_W) &&
           (px_y >= SHIP_Y) && (px_y < SHIP_Y + SHIP_H) &&
           ship_bits[SHIP_COLS-1 - (px_x - ship_x) / SPR_SCALE];
```

14.2 Background: stars and frame

Both title and game screens share the same background. The frame is a 4 px wide border inset 8 px from the edges; its diagonal stripes alternate between magenta and yellow according to bit 3 of (px_x + px_y). The stars come from a cheap hash: the screen is cut into 8 x 8 cells, an XOR of the cell coordinates decides which cells contain a star and where, so the field is fixed (no twinkling) and needs no memory.

src/graphics.v (src/graphics.v, lines 1429-1440)

```
// Border: 4 px wide, inset 8 px from every edge, fixed diagonal
// magenta / yellow stripes.
frm_ph  = px_x[4:0] + px_y[4:0];
frm_hit = ((px_x >= 8) && (px_x < 632) && (px_y >= 8) && (px_y < 472)) &&
          ((px_x < 12) || (px_x >= 628) || (px_y < 12) || (px_y >= 468));
```

```
// Starfield: the screen is cut into 8x8 cells; a cheap XOR hash of
// the cell coordinates (sh) decides which cells hold a star and
// where in the cell it sits. Fixed -- no twinkling.
sh = px_x[9:3] ^ {px_y[6:3], px_y[8:6]} ^ {px_x[5:3], px_x[9:6]};
star_hit = ((px_x[4:3] ^ px_y[4:3]) == 2'b11) && (sh[2:0] == 3'b101) &&
           (px_x[2:0] == sh[5:3]) && (px_y[2:0] == {sh[6], sh[4:3]});
```

14.3 Priority chain

src/graphics.v (src/graphics.v, lines 1454-1477)

```
if (!video_on)
    rgb = COLOR_BLACK;                      // blanking interval
else if (state == S_TITLE)
    rgb = title_rgb;                        // static title screen
else if (game_over)
    rgb = go_hit ? COLOR_MAGENTA : COLOR_BLACK; // static "GAME OVER" message
else if (game_won)
    rgb = go_hit ? COLOR_GREEN : COLOR_BLACK;  // static "GAME WON" message
else if (alien_hit)
    rgb = alien_rgb;
else if (ship_hit)
    rgb = COLOR_WHITE;
else if (bullet_hit)
    rgb = BULLET_COLOR;
else if (alien_bullet_hit || alien_bullet_b_hit)
    rgb = ALIEN_BULLET_COLOR;
else if (hud_hit)
    rgb = COLOR_WHITE;
else if (frm_hit)
    rgb = frm_ph[3] ? COLOR_YELLOW : COLOR_MAGENTA;   // same frame as the title screen
else if (star_hit)
    rgb = COLOR_WHITE;                      // same starfield as the title screen
else
    rgb = COLOR_BLACK;                      // same black sky as the title screen
```

Note that the ordering gives the sprites precedence over the background: an alien or bullet is drawn on top of the stars and the frame. During blanking (!video_on) the colour is forced to black.



Title - MODE EASY (SW1=0,SW0=0) Title - MODE MEDIUM (SW0=1) Title - MODE HARD (SW1=1)

Figure 13. The title screen in each mode. “MODE ...” follows the switches live; everything else is static.

14.4 Output register and the DAC

The mux result goes through one flip-flop on **main** (“fd_1” in the paper) before reaching the pins. It adds one 20 ns clock of delay to the colour (half a pixel) relative to the sync signals, which is invisible, and it prevents glitches from the long combinational mux reaching the monitor. The VGA DAC on the DE2-115 takes 8 bits per channel; each of the three colour bits is replicated to 8 bits, so a colour is either fully off or fully on.

src/main.v (src/main.v, lines 105-107)

```
always @(posedge clk) begin
    rgb <= graphics_rgb_s;
end
```

board/de2115/de2115_top.v (board/de2115/de2115_top.v, lines 108-110)

```
assign VGA_R = {8{rgb[2]}};
assign VGA_G = {8{rgb[1]}};
assign VGA_B = {8{rgb[0]}};
```

The red-channel issue. On the lab monitor/cable the red channel did not reach the screen (red came out black, yellow came out green, white came out cyan). For that reason nothing that must be seen uses pure red: aliens are green or magenta, GAME OVER is magenta, GAME WON is green, MODE HARD is magenta. If a monitor with a working red channel is used, the colours can be changed by editing the localparams in Section 6.1.

15. A complete game, step by step

- **Power on / KEY0:** everything resets. The title screen shows; the LFSRs keep running. The “MODE ...” text follows SW[1:0].
- **Any of KEY1/2/3:** state becomes S_PLAY. The mode (diff), random countdowns and per-alien colours freeze. In HARD mode each alien starts its spawn timer.
- **Each 10 ms (move_tick):** in EASY/MEDIUM the two formation countdowns tick; in HARD every alien ticks its own timers. Bullets move. Shooters A and B count down. Ship moves every 5th tick if a direction key is held.
- **Every pixel, every 40 ns:** the mux decides the colour from the current registers; the picture is redrawn 60 times a second.
- **Player hits an alien:** alien marked defeated, score +2, the alien disappears and can no longer fire.
- **An alien bullet hits the ship:** lives 3 -> 2 -> 1 -> 0. At 0, GAME OVER. **An alien reaches the ship's row:** immediate GAME OVER regardless of lives.
- **All 21 aliens shot:** GAME WON. The screen shows the message and the game is frozen until KEY0.

16. Parameters, speeds and difficulty

All tunable numbers are Verilog parameters at the top of graphics.v. One game tick is 10 ms. For a countdown reloaded with N ticks the step happens every N + 1 ticks. Speeds below are averages computed from the parameters.

Quantity	EASY	MEDIUM	HARD
Motion model	rigid block	rigid block	21 independent agents
Descent wait (reload)	18 + 0..63 ticks	8 + 0..15 ticks	d_min 6/10/15 + 0..15 (per alien class)
Descent step	1,2,3,2 px (avg 2)	1,2,3,2 px (avg 2)	1 or 2 px
Descent speed (avg)	about 4 px/s	about 12 px/s	about 10 / 8 / 6 px/s (3 classes)
Sideways wait (reload)	23 + 0..31 ticks	5 + 0..7 ticks	h_min 3/6/10 + 0..7 (per alien class)
Sideways step	1 px	1 px	1 or 2 px
Sideways speed (avg)	about 2.5 px/s	about 10.5 px/s	about 20 / 14 / 10 px/s
Direction reversal (random)	1 in 256 steps	1 in 256 steps	1 in 64 steps
Fire wait (reload)	171 + 0..255 (about 3 s)	60 + 0..63 (about 0.9 s)	45 + 0..63 (about 0.8 s)
Alien bullet speed	4 px/tick (400 px/s)	6 px/tick (600 px/s)	6 px/tick (600 px/s)
Player bullet speed	4 px/tick	4 px/tick	4 px/tick
Unshot, first alien at ship row (estimate)	about 78 s	about 26 s	about 36 s (from the model)

Sideways reach: in EASY/MEDIUM the block can slide until the leftmost alien is at x = 16 and the rightmost alien's edge at x = 624. In HARD each alien may be anywhere in x 16..586 (edge at 618).

17. Verification, limitations and build notes

17.1 What was checked

- A lint script checks balanced **begin/end**, **case/endcase** and **module/endmodule**, and that no signal is declared twice.
- A tick-level Python model of HARD mode with the real per-alien LFSRs confirmed that aliens never leave the play rectangle, always spawn on the top edge, never all step on the same tick, and that the first unshot alien reaches the ship's row at about 36 s.
- A Python model confirmed that the rigid formation reaches a screen edge in every run, and statistical tests showed the per-alien colour bits are unbiased (about 50 % each) with no neighbour correlation.
- Pictures in this report were generated from the same bitmaps and colour rules, not photographed.

17.2 What has not been checked

- No Quartus compile and no ModelSim run was done from this environment. The five testbenches in sim/tb_main.v are written but unrun.
- Timing at 50 MHz: the pixel mux contains a lot of combinational logic (21 box comparators plus the text and title logic) in front of one register. If Quartus reports a timing failure, the first remedy is to pipeline the colour mux by one more register stage.
- Behaviour on the real monitor beyond what was observed in the lab photos.

17.3 Building it

Open a Quartus project with **de2115_top** as the top-level entity (device EP4CE115F29C7), add the six .v files from src/ plus de2115_top.v, and import board/de2115/de2115_pins.qsf for the pin assignments. For ModelSim use the testbenches in sim/tb_main.v (top modules tb_main, tb_touch, tb_lives, tb_modes, tb_hard).

18. Glossary of signals, parameters and terms

Name	Meaning
clk, CLOCK_50	The single 50 MHz board clock (period 20 ns).
not_reset	Reverse-logic reset input: 0 = reset, 1 = run (driven by debounced KEY0).
move_tick	One-clock pulse every 10 ms used as the clock enable for all game motion.
pixel_tick	Toggle flip-flop that is 1 on every second clock; makes the 25 MHz pixel rate.
h_count, v_count	VGA counters: column (0..799) and line (0..524) of the raster.
px_x, px_y	The pixel being painted now (same as pixel_x, pixel_y; 0..639 and 0..479 when visible).
video_on	1 inside the visible 640x480 area, 0 during blanking.
hsync, vsync	Active-low horizontal and vertical sync pulses to the monitor.
rgb[2:0]	3-bit colour {R,G,B}; each bit drives a whole 8-bit DAC channel.
state	Game FSM register: S_TITLE 0, S_PLAY 1, S_OVER 2, S_WON 3.
mode, mode_n	Raw mode input (SW[1:0]) and its normalised 0/1/2 value.
diff	Difficulty latched at game start: 0 EASY, 1 MEDIUM, 2 HARD.
indep, med	diff[1] (HARD, independent aliens) and diff[0] (MEDIUM timing selector).
any_key	Left OR right OR shoot; used to start the game.
lfsr, lfsr2	32-bit free-running LFSRs (global randomness; lfsr2 serves shooter B and colours).
rnd	The 16-bit LFSR private to each HARD alien.
acol	Per-alien colour bit register: 0 = green, 1 = magenta.
defeated[k]	1 once alien k has been shot.
present[k]	Alien k exists on screen (always 1 in EASY/MEDIUM; in HARD after spawning).
alive[k]	present and not defeated: can be drawn, hit and can fire.
apx_flat, apy_flat	Packed x and y of all 21 aliens (10 bits each); slice k = bits [10*k +: 10].
x_offset, descent_offset	Rigid-formation sideways position (0..288) and sink distance.
MAX_SWEEP	Half the sideways range of the formation: 144 px.
pres, hcls, dcls, hcnt, dcnt, scnt	HARD alien registers: present flag, sideways/descent speed classes, sideways/descent/spawn countdowns.
BOX_X0, BOX_X1, BOX_Y0, BOX_Y1	HARD play rectangle: 16, 618, 52, 440 px.
X_MAX, Y_MAX	Largest legal alien x (586) and y (422) inside that rectangle.
SLOT_W, SPAWN_PAD	Width of an alien's spawn slot (86 px) and the offset inside it (12 px).
ship_x	Left edge of the ship (y is the constant SHIP_Y = 440).
SHIP_STEP	Ship step size, 5 px.
nes_a, nes_b	Two-flop shift register that detects the rising edge of the shoot key.
bullet_active, bullet_x, bullet_y	State of the player's bullet (y is the bullet's bottom edge, drawn 14 px upward).
alien_bullet_*, alien_bullet_b_*	State of shooter A's and shooter B's bullets.
alien_fire_counter(_b)	Countdown (in ticks) to the next shot attempt of shooter A (B).

Name	Meaning
fire_idx	Index 0..20 of the alien picked to fire.
life_lost	Set for one clock when an alien bullet hits the ship.
touch	Set when a live alien has reached the ship's row.
lives_reg, score_reg, level_reg	Lives (3..0), score (adds 2 per hit), level (counts completed levels).
flip_counter	1-bit toggle from the paper; lets the level increment exactly once.
SPR_SCALE	Each sprite bit is drawn as SPR_SCALE x SPR_SCALE screen pixels (2).
FLOOR_Y	Row (468) where alien bullets disappear, just above the bottom frame.
HUD	Heads-up display: the SCORE and LIVES text in the top-right corner.
LFSR	Linear Feedback Shift Register: shift register with XOR feedback, used as a pseudo-random source.
Tap	A register bit that is fed into the XOR of the LFSR feedback.
Debounce	Filtering out the chatter of a mechanical contact.
Metastability	An undefined flip-flop state caused by sampling an asynchronous signal at the wrong instant; avoided with a 2-flop synchroniser.
Clock enable	A one-clock pulse used inside a clocked block instead of creating a slower clock.
Blanking interval	The part of each line/frame that is not displayed; holds the sync pulses.
FP / BP	Front porch / back porch: short margins around the sync pulse.
Priority mux	A chain of if / else-if where the first true condition decides the output.